

# 1. Basic Navigation and Programming

**Course: Python Data Base Management**

**Instructor: Dr. Ujwal Deep Kadiyam**

*Alma mater: IIT Guwahati (PhD)*

**@ DV DATA ANALYTICS**

# What can be done with Python

- Do quick calculations like a smart calculator
- Organize and manage your data easily
- Create simple tools to save time on repetitive tasks
- Read and understand information from Excel or text files
- Make sense of large data with summaries and insights
- Build interactive charts and visuals
- Clean messy data to prepare it for analysis

# How to Install Anaconda

1. Go to the official website: <https://www.anaconda.com>
2. Click on **Download** and choose your operating system
3. Download the **Anaconda Installer** for Python 3.x
4. Run the installer and follow the on-screen instructions
5. Choose "Add Anaconda to PATH" (optional but useful)
6. After installation, open **Anaconda Navigator** or use **Anaconda Prompt**
7. You can now launch **Jupyter Notebook** or other tools for Python development

Home

Environments

Learning

Community

Applications on

base (root)

Channels



CMD.exe Prompt

0.1.1

Run a cmd.exe terminal with your current environment from Navigator activated

Launch



Datalore

Online Data Analysis Tool with smart coding assistance by JetBrains. Edit and run your Python notebooks in the cloud and share them with your team.

Launch



IBM Watson Studio Cloud

IBM Watson Studio Cloud provides you the tools to analyze and visualize data, to cleanse and shape data, to create and train machine learning models. Prepare data and build models, using open source data science tools or visual modeling.

Launch



JupyterLab

3.2.1

An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

Launch



Jupyter Notebook

6.4.5

Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.

Launch



Powershell Prompt

0.0.1

Run a Powershell terminal with your current environment from Navigator activated

Launch



PyCharm Community

2021.3.1

An IDE by JetBrains for pure Python development. Supports code completion, listing, and debugging.

Launch



Qt Console

5.1.1

PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.

Launch



Spyder

5.1.5

Scientific Python Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features

Launch



Glueviz

1.0.0

Multidimensional data visualization across files. Explore relationships within and among related datasets.

Install



Orange 3

3.26.0

Component based data mining framework. Data visualization and data analysis for novice and expert. Interactive workflows with a large toolbox.

Install



PyCharm Professional

A full-fledged IDE by JetBrains for both Scientific and Web Python development. Supports HTML, JS, and SQL.

Install



RStudio

1.1.456

A set of integrated tools designed to help you be more productive with R. Includes R essentials and notebooks.



A full Python IDE directly from the browser

Documentation

Anaconda Blog



27°C  
Mostly cloudy



ENG  
IN  
13:50  
15-08-2022

# Jupyter Notebook vs PyCharm vs Spyder

## Jupyter Notebook

- Web-based interface
- Best for data analysis & visualization
- Code + output + notes together
- Great for beginners and experiments

## PyCharm

- Full-featured IDE by JetBrains
- Ideal for software development
- Advanced debugging, version control
- Best for large Python projects

## Spyder

- Lightweight scientific IDE
- Similar to MATLAB
- Good for data science and research
- Includes variable explorer and plots

# Jupyter Notebook Demo

- **Python Notebook** is an interactive document where you can write, run, and explain Python code.
- **Markdown**: Used to write formatted text, headings, lists, and notes.
- **Code**: Used to write and run Python code that the kernel executes.
- **Raw**: Used to add plain text that is not formatted or executed.

# Jupyter Notebook Kernel – Key Points

- **Kernel = Engine** that runs your code and returns results
- Remembers variables & code until restarted
- Works with many languages (Python, R, Julia...)
- **Restarting** clears memory (like resetting workspace)

**Kernel Menu Actions:**  Restart ·  Interrupt ·  Restart & Run All ·  Change Language

**Tips:** Restart if stuck · Run cells in order after restart · Watch kernel status (top-right) · Save work often

# Libraries in Python for Data Science

## Top 10 Python Libraries



### Pandas

Data analysis and manipulation



### NumPy

Mathematical functions



### Matplotlib

Data visualisations



### SeaBorn

Data visualisations



### Tensorflow

Machine Learning



### Keras

Deep Learning



### SciPy

Scientific computing



### PyTorch

Machine Learning



### Scrapy

Web crawling



### SQLModel

Interact with SQL databases



# Applications of Popular Python Libraries

- **NumPy**: Fast numerical calculations, arrays, and matrix operations
- **Pandas**: Data manipulation: Handling, cleaning, and analyzing structured data (like Excel tables)
- **Matplotlib**: Creating charts and graphs (line, bar, scatter, etc.)
- **Scikit-learn**: Building machine learning models like prediction and classification
- **Flask**: Creating simple web applications and APIs with Python
- **Seaborn**: Advanced and Interactive data visualization
- **STATSMODEL**: Exploratory Data Analysis
- **Keras**: Deep Learning and AI
- **Tensorflow**: Deep Learning and AI

# Library vs Module vs Function

- **Library:** A collection of pre-written code for a specific purpose
  - Example: `pandas` for data handling, `matplotlib` for plotting
  - Library is a collection of modules
- **Module:** A single file containing Python code (can be part of a library)
  - Example: `math`, `random`, or your own file like `mymodule.py`
  - Libraries are made up of one or more **modules**
  - Modules are a collection of functions

# What is a Function?

- A **function** is a reusable block of code that performs a task
- You define a function using `def`, and run it by calling its name
- Can be system defined.
- Example:

```
def greet():  
    print("Hello!")  
greet()  # Output: Hello!
```

# Variables & Data Types in Python

- What are Variables?
  - Think of them as containers or named storage locations.
  - They hold values that can change during program execution.
  - No explicit declaration needed; assignment creates the variable.
- Assigning Values:

```
# Assigning an integer value
age = 30

# Assigning a string value
name = "Alice"

# Variables can be reassigned
age = 31
```

# Variables & Data Types in Python

- **Variable Naming Rules:**
  - Can contain letters, numbers, and underscores.
  - Can start with a letter or an underscore.
  - Case-sensitive (`myVar` is different from `myvar`).
  - Avoid using Python keywords (e.g., `if`, `for`, `class`).

# Variables & Data Types in Python

- What are Data Types?
  - Classify the type of value a variable holds.
  - Determine what operations can be performed on the data.
  - Python is dynamically typed: you don't declare the type, it's inferred.
- Common Built-in Data Types:
  - Numeric:
    - `int`: Integers (e.g., `10`, `-5`)
    - `float`: Floating-point numbers (e.g., `3.14`, `-0.5`)
    - `complex`: Complex numbers (e.g., `1 + 2j`)

# Variables & Data Types in Python

- **Text:**
  - `str`: Strings (sequences of characters, e.g., `"hello"`, `'Python'`)
- **Boolean:**
  - `bool`: Truth values (`True`, `False`)
- **Sequence Types:**
  - `list`: Ordered, mutable collection (e.g., `[1, 2, 3, 'Hello' , 1+3j]`)
  - `tuple`: Ordered, immutable collection (e.g., `(10, 20, 30)`)
  - `range`: Immutable sequence of numbers

# Variables & Data Types in Python

- Mapping Type:
  - `dict`: Unordered, mutable key-value pairs (e.g., `{"name": "Bob", "age": 25}`)
- Set Types:
  - `set`: Unordered collection of **unique** items (e.g., `{1, 2, 3}`). No repetitions.
  - `frozenset`: Immutable version of set
- Checking Data Type:

```
my_variable = "Python"
print(type(my_variable)) # Output: <class 'str'>

number = 123
print(type(number))      # Output: <class 'int'>
```





# Indexing in Python

- Access elements in ordered sequences (**strings, lists, tuples**)
- **Positive Indexing:** starts at `0` from left

```
`my_string="Hello"` → `my_string[0]` → `H`  
`my_list=['apple','banana','cherry']` → `my_list[1]` → `banana`
```

- **Negative Indexing:** starts at `-1` from right

```
`my_string="Python"` → `my_string[-1]` → `n`  
`my_tuple=(10,20,30,40)` → `my_tuple[-2]` → `30`
```

- **IndexError:** accessing a non-existent index raises an error  
`"abc"[3]` → `IndexError`

# `append()` vs `add()` in Python

- `append()` is used with **lists** to add a single item to the end.
- `add()` is used with **sets** to add a unique item.
- `append()` with a list:

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)  # [1, 2, 3, 4]
```

- `add()` with a set:

```
my_set = {1, 2, 3}
my_set.add(4)
print(my_set)  # {1, 2, 3, 4}
```

# `append()` vs `add()` in Python

| Feature           | <code>append()</code>       | <code>add()</code>       |
|-------------------|-----------------------------|--------------------------|
| Used with         | <code>list</code>           | <code>set</code>         |
| Allows duplicates | ✓ Yes                       | ✗ No                     |
| Adds at           | End of list                 | Any position (unordered) |
| Syntax            | <code>list.append(x)</code> | <code>set.add(x)</code>  |

# List: Update, Delete, Append

- Lists are mutable and ordered.

```
# Create a list
fruits = ['apple', 'banana', 'cherry']

# Append
fruits.append('orange') # ['apple', 'banana', 'cherry', 'orange']

# Update
fruits[1] = 'blueberry' # ['apple', 'blueberry', 'cherry', 'orange']

# Delete
del fruits[2]           # ['apple', 'blueberry', 'orange']
```

# Tuple: Update, Delete, Append

- Tuples are immutable — no **direct** update, delete, or append.

```
# Create a tuple
colors = ('red', 'green', 'blue')

# Tuples are immutable!

# To "update", convert to list
temp = list(colors)

# append
temp.append('yellow')

#convert back to tuple
colors = tuple(temp) # ('red', 'green', 'blue', 'yellow')
```

# Set: Update, add and remove

- Sets are unordered and mutable.

```
# Create a set
numbers = {1, 2, 3}

# Append (add)
numbers.add(4) # {1, 2, 3, 4}

# Update (add multiple)
numbers.update([5, 6]) # {1, 2, 3, 4, 5, 6}

# Delete
numbers.remove(2) # {1, 3, 4, 5, 6}
```

# Dictionary: Update, Delete, Append

- Dictionaries store key-value pairs and are mutable.

```
# Create a dictionary
student = {'name': 'Alice', 'age': 20}

# Append / Update
student['grade'] = 'A'    # {'name': 'Alice', 'age': 20, 'grade': 'A'}

# Update existing
student['age'] = 21       # {'name': 'Alice', 'age': 21, 'grade': 'A'}

# Delete
del student['name']      # {'age': 21, 'grade': 'A'}
```

# Operations on Python Data Types

| Operation \ Type | list                                                        | tuple         | set                                                             | dict                                  |
|------------------|-------------------------------------------------------------|---------------|-----------------------------------------------------------------|---------------------------------------|
| append           | ✓ <code>append(x)</code>                                    | ✗ Not allowed | ✗ Use <code>add()</code>                                        | ✗ Use <code>update()</code>           |
| add              | ✗                                                           | ✗             | ✓ <code>add(x)</code>                                           | ✗ Use <code>update()</code>           |
| update           | ✗                                                           | ✗             | ✓ <code>update({...})</code>                                    | ✓ <code>update({...})</code>          |
| delete           | ✓ <code>del</code> , <code>remove</code> , <code>pop</code> | ✗ Immutable   | ✓ <code>remove</code> , <code>discard</code> , <code>pop</code> | ✓ <code>del</code> , <code>pop</code> |



# Creating a DataFrame in Pandas

## Steps:

```
import pandas as pd

data = {
    "ID": [1, 2, 3, 4, 5],
    "Name": ["Alice", "Bob", "Charlie", "Diana", "Ethan"],
    "Score": [85, 90, 78, 92, 88]
}
df = pd.DataFrame(data)
print(df)
```

|   | ID | Name    | Score |
|---|----|---------|-------|
| 0 | 1  | Alice   | 85    |
| 1 | 2  | Bob     | 90    |
| 2 | 3  | Charlie | 78    |
| 3 | 4  | Diana   | 92    |
| 4 | 5  | Ethan   | 88    |

# DF.loc()

- `.loc[]` is used for label-based selection (row/column names, not positions).
- You can select single rows, row ranges, and specific columns using `df.loc[row_label, column_label]`.
- Row ranges are inclusive (e.g., `df.loc[1:3]` includes row 3).

`df.loc[2]`

|                        |         |
|------------------------|---------|
| <code>df.loc[2]</code> |         |
|                        | 2       |
| ID                     | 3       |
| Name                   | Charlie |
| Score                  | 78      |
| dtype: object          |         |

`df.loc[1:3]`

|                          |    |         |       |
|--------------------------|----|---------|-------|
| <code>df.loc[1:3]</code> |    |         |       |
|                          | ID | Name    | Score |
| 1                        | 2  | Bob     | 90    |
| 2                        | 3  | Charlie | 78    |
| 3                        | 4  | Diana   | 92    |

`df.loc[2:4, ['Name', 'ID']]`

|                                          |         |    |
|------------------------------------------|---------|----|
| <code>df.loc[2:4, ['Name', 'ID']]</code> |         |    |
|                                          | Name    | ID |
| 2                                        | Charlie | 3  |
| 3                                        | Diana   | 4  |
| 4                                        | Ethan   | 5  |

`df.loc[:, ['ID', 'Score']]`

|                                         |    |       |
|-----------------------------------------|----|-------|
| <code>df.loc[:, ['ID', 'Score']]</code> |    |       |
|                                         | ID | Score |
| 0                                       | 1  | 85    |
| 1                                       | 2  | 90    |
| 2                                       | 3  | 78    |
| 3                                       | 4  | 92    |
| 4                                       | 5  | 88    |

# Understanding `.iloc[]` in Pandas

- `.iloc[]` is used for **position-based indexing** (by integer positions only).
- Syntax: `df.iloc[row_index, column_index]`. Indices start at 0
- Select a Single Row
  - `df.iloc[2]` #gets row-3.
- Range of rows (start index to end index-1)
  - `df.iloc[1:4]` # rows 1 to 3
- Select Rows and Columns:
  - `df.iloc[1:4, 0:2]`

# `.loc` vs `.iloc` in Pandas

| Feature   | <code>.loc[]</code>                              | <code>.iloc[]</code>                              |
|-----------|--------------------------------------------------|---------------------------------------------------|
| Type      | Label-based indexing                             | Integer (position) based indexing                 |
| Syntax    | <code>df.loc[row_label, col_label]</code>        | <code>df.iloc[row_index, col_index]</code>        |
| Row Range | Inclusive ( <code>df.loc[1:3]</code> includes 3) | Exclusive ( <code>df.iloc[1:3]</code> excludes 3) |
| Use Case  | When using names (like "Name")                   | When using positions (like column 0)              |
| Use       | Filters and subsetting with labels               | Quick access with row/column numbers              |

- Use `.loc` when working with real-world data (e.g., "Customer Name")
- Use `.iloc` when slicing by row/column **number**

# `.loc` vs `.iloc` in Pandas

## `.loc[]`

-  Label-based indexing
-  Uses row/column names
-  Inclusive slicing: `df.loc[1:3]`
-  Example:  
`df.loc[1:3, ['Name', 'Score']]`

## `.iloc[]`

-  Position-based indexing
-  Uses integer positions
-  Exclusive slicing: `df.iloc[1:3]`
-  Example:  
`df.iloc[1:3, 0:2]`

# Create New Column with NaN

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 90, 78]
})

df['Grade'] = np.nan
print(df)
```

The data frame looks like

|   | Name    | Score | Grade |
|---|---------|-------|-------|
| 0 | Alice   | 85    | NaN   |
| 1 | Bob     | 90    | NaN   |
| 2 | Charlie | 78    | NaN   |

# Create New column

```
df['Grade'] = ['B', 'A', 'C']  
print(df)
```

The data frame looks like

|   | Name    | Score | Grade |
|---|---------|-------|-------|
| 0 | Alice   | 85    | B     |
| 1 | Bob     | 90    | A     |
| 2 | Charlie | 78    | C     |

# Deleting Columns in pandas

- There are multiple ways to delete columns from a DataFrame.
- Method 1: Using `del` keyword

- ```
del df['Score']
```

## ◆ Before Deletion

|   | Name  | Score | Grade |
|---|-------|-------|-------|
| 0 | Alice | 85    | B     |
| 1 | Bob   | 90    | A     |

## ◆ After Deletion

|   | Name  | Grade |
|---|-------|-------|
| 0 | Alice | B     |
| 1 | Bob   | A     |



# Deleting Columns in Pandas

```
df.drop('Score', axis=1, inplace=True)
```

| Part                      | Meaning                                                                               |
|---------------------------|---------------------------------------------------------------------------------------|
| <code>df.drop(...)</code> | Calls the <code>drop()</code> function to remove rows or columns from <code>df</code> |
| <code>'Score'</code>      | The name of the column to drop                                                        |
| <code>axis=1</code>       | Tells pandas to drop <b>columns</b> (not rows). Use <code>axis=0</code> for rows      |
| <code>inplace=True</code> | Makes the change <b>permanent</b> in the same DataFrame ( <code>df</code> )           |

- drop multiple columns

```
df.drop(['Score', 'Grade'], axis=1)
```

```
new_df = df.drop('Score', axis=1) # deleting without inplace
```



# Deleting Rows in pandas

## 1 Delete by index

```
df.drop(2, axis=0, inplace=True)
```

## 2 Delete multiple rows

```
df.drop([0, 2], axis=0, inplace=True)
```

## 3 Delete by condition (filtering)

```
df = df[df['Score'] >= 80] # keeps rows where Score >= 80
```

# Rename Columns

- Method 1: Using `rename()` with a dictionary

- ```
df.rename(columns={'Score': 'Marks'}, inplace=True)  
df.rename(columns={'Score': 'Marks', 'Name': 'Student', 'Grade': 'Result', inplace=True)
```

- Method 2: Rename All Columns

- ```
df.columns = ['Student', 'Marks']
```

- Use this to rename all columns at once
- Order and number must match original

- Method 3: Rename Without `inplace=True`

- ```
new_df = df.rename(columns={'Score': 'Marks'})
```

- Creates a new DataFrame, Original df stays unchanged.

# Deleting a DataFrame in pandas

Sometimes you need to **completely remove** a DataFrame

- Method 1: Using `del`

- `del df`

- Deletes the variable `df` entirely from the namespace, cannot access `df` anymore after this line.

- Method 2: None

- `df = None`

- Empties the variable.

- The original DataFrame still exists in memory



# Pandas DataFrame Overview Methods – Quick Reference

| Method                                           | Purpose                                     | Notes                                                               |
|--------------------------------------------------|---------------------------------------------|---------------------------------------------------------------------|
| <code>df.shape</code>                            | Returns (rows, columns)                     | <code>df.shape[0]</code> → rows,<br><code>df.shape[1]</code> → cols |
| <code>df.head()</code> / <code>df.head(n)</code> | First 5 rows (or first <code>n</code> rows) | Preview top records                                                 |
| <code>df.tail()</code> / <code>df.tail(n)</code> | Last 5 rows (or last <code>n</code> rows)   | Preview bottom records                                              |
| <code>df.sample(n)</code>                        | Random <code>n</code> rows                  | Default <code>n=1</code>                                            |
| <code>df.sample(n, random_state=42)</code>       | Random rows with fixed seed                 | Ensures repeatability                                               |



# Pandas DataFrame Overview Methods – Summary

| Method                                                      | Purpose                                          | Example Output                                 |
|-------------------------------------------------------------|--------------------------------------------------|------------------------------------------------|
| <code>df.info()</code>                                      | Structure, non-null counts, dtypes, memory usage | Columns + dtypes + counts                      |
| <code>df.columns</code> /<br><code>df.columns.values</code> | Get column labels (Index / NumPy array)          | <code>Index(['Name', 'Score', 'Grade'])</code> |
| <code>df.dtypes</code>                                      | Data type of each column                         | <code>Score int64, Name object</code>          |
| <code>df.describe()</code>                                  | Summary stats for numeric cols                   | count, mean, std, min, max, quartiles          |
| <code>df.describe(include='all')</code>                     | Summary stats for all col types                  | Adds object columns                            |



# Pandas DataFrame Overview Methods – Summary

| Method                                  | Purpose                                | Example Output             |
|-----------------------------------------|----------------------------------------|----------------------------|
| <code>df.nunique()</code>               | Count unique values per column         | {Name:4, Score:4, Grade:3} |
| <code>df['Grade'].value_counts()</code> | Frequency of unique values in a column | B:2, A:1, C:1              |
| <code>df.isnull()</code>                | Boolean mask for missing values        | True / False table         |
| <code>df.isnull().sum()</code>          | Count missing values per column        | {Score:1, Name:0, Grade:0} |
| <code>df.isnull().sum().sum()</code>    | Total missing values in DataFrame      | 1                          |

# Creating a New DataFrame

- Select columns →

```
df_new = df[["Name", "Score"]]
```

- Filter rows →

```
df_filtered = df[df["Score"] > 80]
```

- Filter + select →

```
df_top = df.loc[df["Score"] > 80, ["Name", "Score"]]
```

- Copy subset →

```
df_copy = df[["Name", "Grade"]].copy()
```



# ⚠ Why `.copy()` Matters

💡 Always use `.copy()` to avoid modifying the original DataFrame unintentionally.

Without `.copy()`:

```
df_top = df[df["Score"] > 80]
df_top["Score"] += 10 # SettingWithCopyWarning`
```

- Changes may affect the original DataFrame
- Can fail silently or trigger warnings

With `.copy()`:

```
df_top = df[df["Score"] > 80].copy()
```

- ✅ Creates a safe, independent DataFrame